APRENDER A PROGRAMAR · INGENIERÍA CIENTÍFICA

Python para Ingeniería
Roadmap intensivo de 3 meses

De la lógica matemática sobre papel a algoritmos, módulos reutilizables, simuladores de ingeniería y un portfolio técnico en GitHub.

Python | Matemáticas | Métodos numéricos | Aeronáutica | Astrodinámica | Portfolio GitHub

> **Objetivo principal:** que al terminar el programa puedas abrir una hoja en blanco, coger un problema de ingeniería, definir sus entradas, estados, ecuaciones, algoritmo, validación y salidas, y convertirlo en código Python estructurado.

Plan de estudio preparado el 18 de agosto de 2026.

1. La idea central

El objetivo no es aprender Python como una colección de comandos. El objetivo es aprender a **convertir matemáticas e ingeniería en software**. Por eso el curso sigue esta cadena:

PROGRAMACIÓN → MATEMÁTICAS → MÉTODOS NUMÉRICOS → INGENIERÍA → AEROESPACIO

Y cada problema se trabajará siguiendo siempre el mismo proceso:

Problema en papel → Algoritmo → Funciones → Módulo → Validación → Plots → Proyecto GitHub

Ejemplo: para una transferencia de Hohmann no empezaremos usando una librería de astrodinámica. Primero implementaremos nosotros la física:

$$v_c = \sqrt{(\mu/r)}$$

$$a_t = (r_1 + r_2)/2$$

$$v_p = \sqrt{[\ \mu(2/r_1 - 1/a_t)\ ]}$$

```
hohmann_transfer.py

circular_velocity()
transfer_semimajor_axis()
transfer_velocity_perigee()
transfer_velocity_apogee()
delta_v_1()
delta_v_2()
transfer_time()
```

Después añadiremos representación, tests y finalmente compararemos la solución con una herramienta profesional. Así se aprende la lógica subyacente y no solamente a utilizar una API.

2. Estructura general de los 3 meses

Ritmo orientativo: **5-7 horas semanales**, repartidas en sesiones de 45-60 minutos cuando sea posible.

Mes	Objetivo	Resultado
Mes 1	Lógica de programación y Python estructurado	Funciones, módulos, arrays, archivos, clases y primeros tests.
Mes 2	Matemática computacional y métodos numéricos	Álgebra, derivadas, integrales, estadística, Monte Carlo y ODE.
Mes 3	Aplicación a aeronáutica y espacio	Actitud, turbojet, dinámica orbital, transferencias y porkchop plots.

3. MES 1 - Aprender a pensar como programador

Semana	Programación	Matemáticas	Proyecto
1	Variables, if, for, while, funciones	Álgebra	Calculadora matemática
2	Listas, tuplas, diccionarios, arrays	Vectores y matrices	Biblioteca vectorial
3	Funciones, módulos, imports	Trigonometría y coordenadas	Coordinate Transform Toolkit
4	Clases, archivos, excepciones	Geometría/ingeniería	Mini biblioteca científica

Semana 1 - Lógica, iteración y funciones

No basta con saber qué hace un `for`; hay que aprender **cuándo un problema requiere una iteración**.

$$S = \Sigma_{i=1..N}\, i^2$$

```
total = 0
for i in range(1, N + 1):
    total += i**2
```

Después se convierte el algoritmo en una función, se prueba y se discuten alternativas.

```
def sum_squares(n):
    ...

assert sum_squares(3) == 14
```

Práctica complementaria: Exercism y Project Euler.

Semana 2 - Estructuras de datos, vectores y matrices

Introducción progresiva a NumPy, pero evitando inicialmente que las librerías oculten los algoritmos. Por ejemplo, antes de usar `numpy.linalg.solve` implementaremos una eliminación de Gauss sencilla.

$$A\,x = b$$

```
import numpy as np
x = np.linalg.solve(A, b)
```

El objetivo es entender qué está haciendo la librería y cuándo utilizarla.

Semana 3 - Funciones, módulos y sistemas de coordenadas

```
coordinate_transforms/
├── rotations.py
├── spherical.py
├── frames.py
├── constants.py
└── tests/
```

Implementaremos las matrices de rotación $R_x(\varphi)$, $R_y(\theta)$, $R_z(\psi)$ y funciones como:

```
def rot_x(phi):
    ...

def rot_y(theta):
    ...

def rot_z(psi):
    ...

def euler_to_rotation_matrix(phi, theta, psi):
    ...
```

> **Proyecto:** Coordinate Transform Toolkit - Cartesian ↔ Spherical, NED ↔ Body, ECEF ↔ local frame y Euler angles ↔ rotation matrix.

Semana 4 - Clases y organización del software

Una clase tendrá sentido cuando exista una relación clara entre **datos, estado y comportamiento**.

```
class Turbojet:
    def __init__(self, compressor_ratio, turbine_efficiency):
        self.compressor_ratio = compressor_ratio
        self.turbine_efficiency = turbine_efficiency

    def compressor_exit_temperature(self):
        ...

    def thrust(self):
        ...
```

También podremos representar objetos físicos como una órbita o un vehículo.

4. MES 2 - Matemática computacional

Semana	Matemáticas	Programación	Proyecto
5	Álgebra lineal y ecuaciones	NumPy / SciPy	Numerical Toolbox
6	Derivadas e integrales	SymPy / SciPy	Calculus Toolkit
7	Estadística	pandas / Monte Carlo	Uncertainty Simulator
8	ODE	Euler / RK4 / solve_ivp	Dynamics Simulator

Semana 5 - Álgebra numérica

Implementaciones propias antes de utilizar las librerías:

- Resolución de sistemas A x = b.
- Bisección.
- Newton-Raphson.
- Interpolación.
- Mínimos cuadrados.

$$x_{n+1} = x_n - f(x_n)/f'(x_n)$$

$$\hat{x} = (A^T A)^{-1} A^T b$$

Semana 6 - Cálculo diferencial e integral

$$f'(x) \approx [f(x+h) - f(x-h)] / (2h)$$

Derivación numérica, rectángulos, trapecios y Simpson. Después introducción a `scipy.integrate` y a SymPy para cálculo simbólico.

Semana 7 - Estadística y Monte Carlo

Media, desviación estándar, covarianza, correlación y distribuciones, aplicadas a simulaciones de incertidumbre.

> **Mini proyecto:** dispersión de alcance de un proyectil variando velocidad, ángulo, masa, drag y viento en miles de simulaciones.

```
for simulation in range(N):
    ...

# Outputs
mean_range
standard_deviation
percentile_95
```

Semana 8 - Ecuaciones diferenciales ordinarias

$$dy/dt = f(t,y)$$

Implementaremos primero Euler, luego RK2 y finalmente RK4.

```
def rk4_step(f, t, y, dt):
    ...
```

Después se comparará con `scipy.integrate.solve_ivp`.

> **Proyecto:** Falling Body Simulator con drag y salidas `altitude.dat`, `velocity.dat` y `acceleration.dat`.

5. MES 3 - Aeronáutica y espacio

Semana	Tema	Proyecto
9	Avión y actitud	Aircraft Attitude Simulator
10	Propulsión	Turbojet Performance Simulator
11	Astrodinámica	Orbital Mechanics Toolkit
12	Misión espacial	Launcher / Orbit Mission Simulator + Porkchop

Semana 9 - Flight Mechanics

Ángulos de Euler φ, θ, ψ; matrices de transformación; progresivamente estados, velocidades, fuerzas, momentos y tasas angulares.

> **Aircraft-Attitude-Simulator**: Euler angles, rotation matrices, quaternions, body/NED transformations, IMU angular rates y visualización 3D.

Semana 10 - Turbojet

Atmosphere → Intake → Compressor → Combustion Chamber → Turbine → Nozzle

Cálculo de estaciones termodinámicas y prestaciones.

$$F = \dot{m}(V_e - V_0)$$

$$TSFC = \dot{m}_f / F$$

Estudios paramétricos frente a altitud, Mach, relación de compresión, TIT y eficiencias.

Semana 11 - Astrodinámica

Constantes físicas, problema de dos cuerpos, elementos orbitales, conversión Cartesian ↔ Keplerian, Hohmann, cambios de inclinación, velocidad de escape, esfera de influencia y Lambert.

$$\ddot{r} = -\mu\, r / |r|^3$$

Estado de propagación:

$$x = [x, y, z, v_x, v_y, v_z]$$

Semana 12 - Porkchop plot y misión interplanetaria

Se combinarán fechas de salida y llegada, propagación, problema de Lambert, C3, v∞ y ΔV.

```
for departure_date in departures:
    for arrival_date in arrivals:
        ...
        # Solve Lambert
        # Compute C3 / v_inf / delta-v
```

> **Resultado:** Earth → Mars Porkchop Plot como proyecto de portfolio.

6. Después de los tres meses - PDE y cálculo continuo

La siguiente etapa será introducir ecuaciones diferenciales parciales mediante diferencias finitas.

$$\partial T/\partial t = \alpha \, \partial^2 T/\partial x^2$$

$$T_i^{n+1} = T_i^n + \alpha(\Delta t/\Delta x^2)(T_{i+1}^n - 2T_i^n + T_{i-1}^n)$$

> **Proyecto inicial:** Heat Transfer Through an Aircraft Panel. Después: heat equation 2D, wave equation y Poisson.

7. Recursos recomendados

MIT OpenCourseWare - Introduction to CS and Programming Using Python
Especialmente útil para pensamiento computacional, branching, iteration, funciones y descomposición de problemas.
https://ocw.mit.edu/courses/6-100l-introduction-to-cs-and-programming-using-python-fall-2022/

Harvard CS50P - Introduction to Programming with Python
Muy práctico para condicionales, loops, excepciones, librerías, tests, file I/O y orientación a objetos.
https://cs50.harvard.edu/python/

Exercism - Python Track
Banco de ejercicios cortos para adquirir soltura, practicar conceptos y acostumbrarse a tests.
https://exercism.org/tracks/python

Project Euler
Problemas de razonamiento matemático y programación; especialmente útil una vez dominados condicionales, loops y funciones.
https://projecteuler.net/archives

Berkeley - Python Programming and Numerical Methods
Recurso especialmente alineado con ingeniería: programación, métodos numéricos, álgebra, ODE, integración, interpolación y más.
https://pythonnumericalmethods.berkeley.edu/

Documentación oficial
Python: https://docs.python.org/3/tutorial/
NumPy: https://numpy.org/doc/stable/user/absolute_beginners.html
SciPy: https://docs.scipy.org/doc/scipy/
SymPy: https://docs.sympy.org/latest/
pandas: https://pandas.pydata.org/docs/getting_started/
Matplotlib: https://matplotlib.org/stable/gallery/

Vídeo y formación aeroespacial
Como complemento para métodos numéricos y astrodinámica, el material de Alfonso González es útil para ver implementaciones de integración orbital, Runge-Kutta y mecánica orbital.
https://www.youtube.com/@AlfonsoGonzalezSpaceEngineering

Testing con pytest
Lo introduciremos cuando empecemos a convertir ejercicios en módulos reutilizables y queramos validar automáticamente resultados y casos límite.
https://docs.pytest.org/en/stable/getting-started.html

Astropy Coordinates
Referencia profesional para comparar posteriormente nuestras propias conversiones y representaciones de coordenadas.
https://docs.astropy.org/en/stable/coordinates/

8. Repositorios y herramientas profesionales para estudiar más adelante

Proyecto	Uso	Cuándo estudiarlo
RocketPy	Trayectoria y simulación de cohetes	Después de nuestro simulador 1D/2D
TudatPy	Astrodinámica, propagación, estimación y diseño de misión	Después de Two-Body y elementos orbitales
AeroSandbox	Aerodinámica, diseño y optimización aeronáutica	Una vez sólidos NumPy/SciPy y arquitectura de código
RotorPy	Dinámica 6-DoF de multirrotores	Después de actitud y ODE
JSBSim	Flight dynamics	Después del simulador de actitud
pyCycle	Modelado termodinámico de motores	Después del turbojet propio

Enlaces de referencia: RocketPy - github.com/RocketPy-Team/RocketPy · TudatPy - docs.tudat.space · AeroSandbox - github.com/peterdsharpe/AeroSandbox · RotorPy - github.com/spencerfolk/rotorpy · JSBSim - jsbsim-team.github.io/jsbsim/python/ · pyCycle - github.com/OpenMDAO/pyCycle.

9. Arquitectura objetivo de un proyecto GitHub

```
orbital-mechanics-python/
├── README.md
├── requirements.txt
├── src/
│   ├── constants.py
│   ├── dynamics.py
│   ├── integrators.py
│   ├── orbital_elements.py
│   ├── maneuvers.py
│   └── plotting.py
├── tests/
│   ├── test_integrators.py
│   ├── test_orbital_elements.py
│   └── test_maneuvers.py
├── examples/
│   ├── circular_orbit.py
│   ├── eccentric_orbit.py
│   └── hohmann_transfer.py
├── notebooks/
│   └── earth_mars_analysis.ipynb
├── data/
└── figures/
```

El objetivo será acostumbrarnos desde el principio a separar cálculos, entrada/salida, representación, ejemplos y tests.

10. Los cuatro repositorios principales del portfolio

Repositorio	Contenido
Python-Numerical-Methods	Álgebra, root finding, derivadas, integrales, RK4 y Monte Carlo.
Aircraft-Dynamics-Python	Coordenadas, actitud, atmósfera y performance.
Turbojet-Performance-Simulator	Ciclo termodinámico, thrust, TSFC y estudios paramétricos.
Orbital-Mechanics-Python	Propagación, elementos orbitales, Hohmann, Lambert y porkchop.

Proyecto opcional adicional: **Rocket-Flight-Simulator**, incorporando masa variable, thrust, drag, gravedad y atmósfera, y evolucionando desde 1D hasta 2D/3D.

11. Cómo pasar de una ecuación a un programa

Antes de escribir código se redactará una ficha de diseño mínima.

```
PROBLEM
Calculate the trajectory of a projectile with drag.

INPUTS
mass, Cd, area, rho, v0, angle

STATES
x, z, vx, vz

EQUATIONS
dx/dt  = vx
dz/dt  = vz
dvx/dt = ...
dvz/dt = ...

ALGORITHM
1. Read parameters
2. Create initial state
3. Compute derivatives
4. Integrate state
5. Stop when z <= 0
6. Store results
7. Plot trajectory

OUTPUTS
time, position, velocity, maximum altitude, range
```

> **Hábito fundamental:** no comenzar programando. Primero traducir el problema físico a entradas, estados, ecuaciones, pasos del algoritmo y salidas.

12. Método de tutoría y revisión

Los ejercicios se trabajarán con un sistema de revisión similar al de un equipo técnico. Cuando presentes tu código se revisarán:

- Lógica y corrección matemática.
- Estilo Python y nombres.
- Casos límite.
- Eficiencia y legibilidad.
- Modularidad.
- Testing.
- Qué concepto conviene entrenar después.

La intención es evitar darte inmediatamente la solución completa cuando el objetivo sea que desarrolles la lógica por ti mismo.

13. Estructura del Bootcamp Level 1

1. Condicionales.
2. Loops.
3. Funciones.

4. Lists, dictionaries y arrays.
5. Scope.
6. Errores y excepciones.
7. NumPy.
8. Matplotlib.
9. Archivos.
10. Módulos.
11. Classes.
12. Testing.

Cada tema seguirá el mismo patrón:

> **Theory → 2 ejercicios simples → 2 ejercicios matemáticos → 1 ejercicio de ingeniería → 1 mini proyecto**

Ejemplo para `for`:

> **1+2+...+N → factorial → serie de Taylor → trayectoria iterativa → Monte Carlo**

14. Qué significa completar con éxito el programa

La meta no es poder decir “sé Python”. La meta es que puedas mirar una ecuación, una hoja de cálculo o un problema de ingeniería y comenzar a estructurarlo computacionalmente con seguridad.

> **Resultado buscado:** que herramientas como RocketPy, Tudat, filtros Kalman, motores, estructuras o cualquier librería científica dejen de parecer cajas negras y se conviertan en software construido con bloques que tú ya sabes diseñar y validar.

Este documento es el mapa de referencia. Las clases prácticas, ejercicios, soluciones parciales, revisiones de código y proyectos se desarrollarán progresivamente para evitar aprendizaje pasivo.